ATIF-Gemma: An Open Empirical Diagnostic Corpus and Failure Taxonomy for Small Open-Weight Developer Agents

Karandeep Singh Sodhi

Independent Researcher

karandeep.sodhi2504a@gmail.com

October 27, 2026

Track: Google — The Gemma 4 Developer Agent Competition (Paper Track: Best New Resource)

Author Note & Resource Manifest

Karandeep Singh Sodhi is an independent researcher.

Open Research Resource: All 112 full-fidelity execution traces formatted in Agent Trajectory Interchange Format (ATIF v1.7), ground-truth reference specifications, metadata index, diagnostic validation scripts, and the companion `atif_gemma` Python package are openly released on Kaggle and GitHub under the Creative Commons Attribution 4.0 International (CC BY 4.0) license.

Kaggle Dataset: https://www.kaggle.com/datasets/karandeepsinghsodhi/atif-gemma-swe-agent-traces
GitHub Repository: https://github.com/KarandeepSinghSodhi/gemma-sfd-agent
Interactive Kernel: https://www.kaggle.com/code/karandeepsinghsodhi/atif-gemma-diagnostic-exploration

Disclosures: The author declares no competing financial or non-financial interests.

Correspondence: Correspondence concerning this article should be addressed to Karandeep Singh Sodhi, Email: karandeep.sodhi2504a@gmail.com.

Abstract

Current software engineering (SWE) agent benchmarks report scalar pass/fail resolution rates, obscuring the multi-stage dynamics of agent failure. For small, quantized open-weight models such as Gemma 4 E4B, understanding why agents plateau at low resolution rates (median 5.0% across 197 teams on the official Small Model Prize leaderboard) is hindered by a complete absence of publicly available, fine-grained execution trace data. We introduce **ATIF-Gemma**, an open empirical diagnostic corpus of 112 full-fidelity multi-turn execution traces comprising 1,518 LLM calls, 1,182 tool invocations, and 11,706,178 logged tokens generated across 32 paired SWE-bench tasks on 4x NVIDIA L4 GPUs inside the official Google ADK and `swegemma` execution harness.

Formatted in the Agent Trajectory Interchange Format (ATIF v1.7), ATIF-Gemma cross-references every step against ground-truth repository patches and held-out test patches. We establish an empirical 4-way Phase 2 exit code failure taxonomy: while 92.0% of runs (103/112) successfully synthesize a non-empty patch, 32.1% (36/112) exit with collection/import errors (`exit_code = 2`) due to sandbox dependency absences, global virtual environment shadowing, and detached syntax blocks; 50.0% (56/112) exit with test assertion failures (`exit_code = 1`) due to partial localization or imprecise edits; 8.0% (9/112) produce no patch due to budget exhaustion (`exit_code = -1`); 2.7% (3/112) experience verification timeouts (`exit_code = 124`); and 7.1% (8/112) achieve full resolution (`exit_code = 0`). Furthermore, we release an Actionable Remediation Playbook and the pip-installable `atif_gemma` diagnostic Python toolkit, providing the developer agent research community with an empirical foundation to diagnose and eliminate failure modes in small open-weight coding agents.

Keywords: agent trajectory corpus, SWE-bench, Gemma 4, failure taxonomy, ATIF v1.7, open-weight agents, error diagnostics, remediation playbook

1. Introduction & The Diagnostic Vacuum in SWE Benchmarking

Autonomous software engineering agents powered by large language models have achieved notable progress on repository-level repair benchmarks such as SWE-bench [1]. However, benchmarking in this domain remains almost universally **outcome-centric**: evaluations report a single scalar accuracy metric (the proportion of tasks resolved), discarding the rich, high-dimensional trajectory of decisions, tool calls, token expansions, and environmental interactions that preceded the outcome.

For frontier models (e.g. Claude 3.5 Sonnet, GPT-4o) and large open-weight models (Gemma 4 31B), high resolution rates (22.0%–24.0% on the Google Gemma 4 challenge) allow researchers to treat the agentic loop as an effective black box. In contrast, for compact, 4-bit quantized open-weight models suitable for local developer workstations (such as `gemma-4-e4b-it-qat-w4a16-ct`), scalar benchmarking reveals an acute capability cliff without explaining its origins. Across all $N = 197$ participating teams on the official Small Model Prize leaderboard (ryanholbrook/gemma-4-developer-agent-small-model-leaderboard), public resolution scores range from 0.0% to 10.0%, with a median of exactly 5.0%, a mean of 4.23%, and 183 of 197 teams (92.9%) scoring $\leq 6.0\%$.

When an agent fails, developers currently cannot discern whether failure stemmed from:

1. **Upstream Localization Failure**: Did the agent search the wrong directories or read irrelevant modules?
2. **Context Saturation & Syntax Corruption**: Did prompt bloat degrade the model's instruction-following fidelity, causing malformed tool arguments or detached code blocks?
3. **Environmental & Sandbox Artifacts**: Did the patch fail held-out tests due to missing third-party packages in the offline sandbox, or global virtual environment packages shadowing local repository source files?
4. **Tool Exhaustion**: Did the agent exhaust its global turn, time, or tool limits before applying an edit?

To bridge this diagnostic gap, we introduce **ATIF-Gemma**, an open, standardized, full-fidelity diagnostic trajectory corpus and software toolkit for small open-weight developer agents. We provide 112 complete execution traces totaling 11,706,178 tokens, 1,518 LLM turns, and 1,182 tool invocations collected on real 4x NVIDIA L4 GPUs across 32 unique benchmark tasks, structured under the Agent Trajectory Interchange Format (ATIF v1.7).

2. Corpus Architecture & ATIF v1.7 Schema Specifications

Corpus Partition	Traces	Tasks	LLM Calls	Tool Calls	Total Tokens
Cohort 1 (Canonical Paired)	24	8	304	236	2,270,463
Cohort 2 (Stratified Random)	24	8	339	269	2,720,322
Cohort 1 Rerun (Reproducibility)	16	8	204	155	1,387,758
Cohort 3 (Stratified Disjoint)	24	8	349	270	2,771,960
Cohort 4 (Stratified Disjoint)	24	8	322	252	2,555,675
Total ATIF-Gemma Corpus	**112**	**32**	**1,518**	**1,182**	**11,706,178**

Table 1: Summary composition of the ATIF-Gemma diagnostic trace corpus across five cohorts on 4x NVIDIA L4 GPUs.

The corpus covers 32 unique SWE-bench instances spanning four major Python repositories: `fastapi` (51 runs), `rich` (37 runs), `requests` (19 runs), and `httpx` (5 runs). All experiments were served using vLLM 0.19.1 under tensor parallel 4 (`tp=4`, 96 GB aggregate VRAM) hosting `gemma-4-e4b-it-qat-w4a16-ct` with Google ADK 1.36.1 and `swegemma`.

Each trace conforms strictly to the **ATIF v1.7 JSON Schema**:

- **Metadata Block**: Records `instance_id`, `repo`, `base_commit`, `agent_config_name`, vLLM sampling parameters (`temperature=0.0`, `seed=42`, `max_output_tokens=8192`, `thinking_budget=1536`), and hardware fingerprints.
- **Step Sequence**: Sequential array of interaction turns containing:
  - `metrics`: Exact per-call `prompt_tokens`, `completion_tokens`, and `total_tokens` reported directly by the vLLM engine.
  - `tool_calls`: Function name, input argument JSON dictionary, and stage author tag (`extra.author`).
  - `observation`: Raw tool execution stdout/stderr response, JSON-parsed error status, and error classification types (`FileEditError`, `CommandError`, `FileReadError`, `BudgetExceeded`).
  - `thought`: Full model reasoning trace emitted within the thinking budget.
- **Final Metrics**: Captures cumulative token counts, total duration in seconds, agent patch string, and Phase 2 test execution exit codes.

3. The 4-Way Phase 2 Exit Code Failure Taxonomy

In SWE-bench, patch verification is executed in an isolated verification sandbox using `pytest`. In ATIF-Gemma, cross-referencing agent traces against Phase 2 test runner outputs reveals that treating unresolved runs as a monolithic failure category conceals fundamentally different failure mechanics. We define an empirical 4-way taxonomy:

Category	Exit Code	Count (%)	Empirical Mechanism & Observable Symptoms
Resolved	0	8 (7.1%)	Patch passes all Phase 2 held-out unit tests with zero regressions.
Assertion Failure	1	56 (50.0%)	Patch syntactically valid and executes under pytest, but fails assertions due to wrong logic or incomplete multi-file edits.
Collection Error	2	36 (32.1%)	Pytest crashes during test collection prior to assertion evaluation (ModuleNotFoundError, SyntaxError, venv shadowing).
Budget / Timeout	−1	9 (8.0%)	Agent exhausted 15 tool calls or 2.5 min wall time; 0 patch characters submitted.
Verification Timeout	124	3 (2.7%)	Verification harness timed out (120s limit) due to daemon process or infinite loop in test runner.

Table 2: Empirical Phase 2 exit code distribution across all 112 ATIF-Gemma execution traces.

3.1 Analysis of Collection / Import Errors (Exit Code 2: 32.1%)

A critical finding enabled by ATIF-Gemma is that **nearly one-third of all agent runs (36/112) failed before unit tests could even execute**. Our trace cross-referencing identifies three distinct sub-mechanisms:

1. **Missing Sandbox Dependencies**: In tasks such as `fastapi_14361` and `fastapi_14459`, the held-out `test_patch` imports `inline_snapshot`. Because `inline_snapshot` was not packaged into the offline wheelhouse, pytest terminates with `ModuleNotFoundError` during test collection across all evaluated configurations.
2. **Environment Shadowing**: On `httpx_3672`, the offline sandbox environment resolved `import httpx` to `/usr/local/lib/python3.12/dist-packages/httpx` (pre-installed system package) rather than the local repository source at `/workspace/src/httpx`, rendering source edits invisible during test collection.
3. **Syntactic Corruption**: In `fastapi_14099`, `PatchAgent` inserted an unindented loop that left a detached `else:` block at line 259, raising `SyntaxError: invalid syntax` during collection.

3.2 Ground-Truth Resolved Runs

Across the 112 runs, 8 runs successfully resolved held-out tests across 4 distinct tasks:

- `rich_4077`: Resolved by `sfd` (5 tools, 40.3s), `sfd_rerun` (6 tools, 41.8s), and `flat_controlled_rerun` (5 tools, 49.2s).
- `fastapi_9753`: Resolved by `flat_starter` (7 tools, 46.1s).
- `fastapi_14077`: Resolved by both `sfd` (6 tools, 43.9s) and `flat_controlled` (12 tools, 67.5s). Notably, SFD resolved the task with 50% fewer tool calls and 35% faster runtime.
- `rich_3894`: Resolved by `flat_starter` (3 tools, 26.0s) and `flat_controlled` (5 tools, 30.7s).

4. Quantitative Audit Across Agent Architectures

ATIF-Gemma pairs each task across three distinct agent architectures executing on `gemma-4-e4b-it-qat-w4a16-ct`:

Configuration	Runs	Resolved	Mean Peak P_t	Mean Prompt	Mean Duration
Official Starter (flat_starter)	32	2/32 (6.2%)	13,359.7	105,817.6	94.2s
Single-Agent (flat_controlled)	32	2/32 (6.2%)	13,959.0	118,173.0	90.8s
State-Filtered Deliberation (sfd)	32	2/32 (6.2%)	10,907.5	80,375.7	86.6s
Rerun: flat_controlled	8	1/8 (12.5%)	12,557.5	93,135.6	107.4s
Rerun: sfd	8	1/8 (12.5%)	9,306.4	67,710.1	96.4s

Table 3: Architectural comparison of token consumption, peak prompt lengths, and execution speed across all 112 ATIF-Gemma runs.

4.1 Sub-Agent Bypassing in the Starter Kit

ATIF-Gemma provides the first empirical trace evidence of how small models handle discretionary sub-agents. The official starter kit wraps `code_analyzer` as an `AgentTool`. Across all 32 `flat_starter` traces:

- Out of 336 tool invocations, the root agent delegated to `code_analyzer` only **7 times (2.1%)**.
- The model completely **bypassed `code_analyzer` on 29 of 32 tasks (90.6%)**, opting instead to read large files directly into its primary context.

4.2 Cross-Stage Prompt Resets

In `sfd` traces, configuring `include_contents: "none"` on downstream stages resets entry prompts from a mean TriageAgent peak of **10,922 tokens** down to **1,500 tokens** at PatchAgent entry (an **−86.3% cross-boundary reset**), reducing cumulative prompt tokens by **32.0%** relative to single-agent baselines across all 32 tasks (mean 80,375.7 vs 118,173.0 tokens).

5. Actionable Remediation Playbook for Developer Agents

A primary contribution of the ATIF-Gemma corpus is converting diagnostic failure classifications into an actionable, production-ready engineering playbook for developers building small coding agents:

Failure Mode	Prevalence	Recommended Production Remediation Pattern
Collection / Import Crash	36/112 (32.1%)	Enforce `PYTHONPATH=/workspace/src:/workspace` priority in test execution commands and execute pre-flight dependency imports with `python -c` before running test suites.
Syntax / Diff Corruption	100 FileEditErrors	Mandate a strict `python -m py_compile <modified_file>` verification check before calling `submit_patch` to prevent broken indentation or detached blocks.
Cascading Localization	15% of pipeline runs	Introduce a two-way validation loop where Stage 2 verifies that the localized symbol exists in the problem description before executing an edit.
Prompt Bloat / Memory Leak	100% of flat agents	Implement stage-isolated context pruning via `include_contents: "none"` in Google ADK, resetting prompt context by 86% across pipeline boundaries.

Table 4: Actionable developer agent remediation playbook derived from empirical ATIF-Gemma failure traces.

6. The `atif_gemma` Python Package & CLI

Along with the dataset, we release a standalone, pip-installable Python package (`atif-gemma`):

- `atif_gemma.audit`: Automatically parses any ATIF v1.7 trajectory directory, extracting token trajectories, tool error frequencies, and exit-code failure buckets.
- `atif_gemma.taxonomy`: Deterministic classifier for Phase 2 test runner exit codes.
- `atif_gemma.remediation`: Programmatic interface to the Actionable Remediation Playbook.
- `atif-gemma` CLI: Command-line tool providing instant diagnostics:

```
python -m atif_gemma.cli audit traces/
python -m atif_gemma.cli playbook
```

7. Limitations & Threats to Validity

ATIF-Gemma records 112 paired runs on 32 tasks ($n = 32$). While statistically stratified across four major repositories, evaluating all 129 SWE-bench instances would require 500 GPU hours. Second, tests were conducted specifically on 4-bit quantized Gemma 4 E4B; while representative of small open-weight developer agents, frontier models may display different failure distributions.

8. Conclusion & Resource Availability

ATIF-Gemma establishes an open, reproducible, full-fidelity diagnostic foundation for the small-model coding agent community. By moving beyond scalar pass/fail metrics to an open 112-trace ATIF v1.7 corpus, a 4-way Phase 2 exit code taxonomy, an Actionable Remediation Playbook, and an open diagnostic Python library, this resource enables researchers to target the real bottlenecks of small developer agents: sandbox collection errors, context bloat, and localization drift.

Dataset URL: https://www.kaggle.com/datasets/karandeepsinghsodhi/atif-gemma-swe-agent-traces
Code Repository: https://github.com/KarandeepSinghSodhi/gemma-sfd-agent
Interactive Exploration Kernel: https://www.kaggle.com/code/karandeepsinghsodhi/atif-gemma-diagnostic-exploration

References

[1] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” *International Conference on Learning Representations (ICLR)*, 2024.